

Plateforme Mastering Live Debix

Spécification Phase 1a.2 — V1.5__glm

Tap HP_Monitor post-effets OUT

Proposition GLM Option A : m4-only, PIPE 7 autonome TIMER + VIRTUAL_WIDGET +
PCM_DUPLEX_ADD

Document de référence technique

2026-04-24 — rev V1.5__glm (candidate)

Table des matières

1	Ce qui change depuis V1.2	2
1.1	Modifications V1.5__glm vs V1.2	2
1.2	Rationale GLM	2
1.3	Risques identifiés (à valider par investigation)	3
1.4	Plan de test	3
2	Ce qui change depuis V1.1	3
2.1	Bug ordre m4 corrigé en V1.2	4
2.2	Points V1.1 confirmés par 4 workers (kimi, minimax, claude-code, glm)	4
3	Ce qui change depuis V1	4
3.1	Bugs V1 corrigés en V1.1	5
3.2	Points V1 confirmés unanimement (pas de changement)	5
4	Contexte — Phase 1a.1 validée	5
5	Architecture Phase 1a.2	5
5.1	Schéma cible	5
5.2	Mécanisme piggyback scheduling	6
5.3	Mécanisme MUXDEMUX en fin de pipeline playback	6
6	Phasage Phase 1a.2	7
7	Squelette m4 Phase 1a.2 V1	7
7.1	sof-imx8mp-tac5212-pipe-effects-playback-tap.m4 (modifié)	7
7.2	sof-imx8mp-tac5212-pipe-hp-monitor-capture.m4 (nouveau)	8
7.3	sof-imx8mp-tac5212-masterV42-hpmon.m4 (top-level étendu)	9
7.4	m4/demux_route_hpmon.m4 (nouveau blob route matrix)	10
8	Tests Phase 1a.2	11
9	Questions pour l'investigation critic V1	11
10	Livrables Phase 1a.2	12
11	Protocole	12

1 Ce qui change depuis V1.2

V1.2 testée sur board a échoué au runtime avec deux bugs distincts :

- `aplay SAI_Playback` : -22 EINVAL trigger START (IPC 0x60040000). Cause racine firmware = asymétrie IPC3 entre `pipeline_comp_prepare` (filtre par direction via `comp_same_dir`) et `pipeline_comp_trigger` (filtre par sched via `is_same_sched`). Piggyback cross-direction avec `sched_comp` partagé de type DAI = PIPE 7 jamais prepared mais reçoit trigger → EINVAL.
- `arecord HP_Monitor` : "no backend DAIs enabled for HP_Monitor". Cause = ASoC DPCM kernel ne trouve pas de BE DAI capture pour le FE HP_Monitor (PIPE 7 dailess, chaîne DAPM remonte vers DAI playback).

Investigation V1.4 (6 workers) a proposé plusieurs fixes. Cette V1.5_glm teste l'**Option A proposée par GLM** (la seule solution m4-only dans les findings) avant d'envisager un patch kernel.

1.1 Modifications V1.5_glm vs V1.2

Modif	Changement V1.5_glm
M1 — PIPE 7 autonome	Retirer le 13e arg <code>PIPELINE_PLAYBACK_SCHED_COMP_6</code> de <code>PIPELINE_PCM_ADD PIPE 7</code> . Changer 12e arg en <code>SCHEDULE_TIME_DOMAIN_TIMER</code> . Ainsi <code>is_same_sched = false</code> entre PIPE 6 et PIPE 7 → le trigger ne traverse plus cross-pipeline → bug -22 neutralisé.
M2 — <code>W_PIPELINE</code> local	Dans <code>pipe-hp-monitor-capture.m4</code> , changer <code>W_PIPELINE(SCHED_COMP, ...)</code> en <code>W_PIPELINE(N_PCMC(PCM_ID), ...)</code> . Ajouter <code>indir(define, PIPELINE_SCHED_COMP_7, N_PCMC(PCM_ID))</code> pour exporter un <code>sched_comp</code> de type HOST (pattern <code>pipe-masterchain.m4:146-147</code>).
M3 — <code>VIRTUAL_WIDGET</code>	Ajouter <code>VIRTUAL_WIDGET(OUT_MONITOR_TAP PIPELINE_ID, out_drv, PIPELINE_ID)</code> dans <code>pipe-hp-monitor-capture.m4</code> pour servir de proxy BE DAPM côté kernel ASoC DPCM. Ajouter <code>dapm(OUT_MONITOR_TAP PIPELINE_ID, N_BUFFER(0))</code> en fin de graph.
M4 — <code>PCM_DUPLEX_ADD</code>	Dans <code>masterV42-hpmon.m4</code> , remplacer les 3 <code>PCM_CAPTURE_ADD/PCM_PLAYBACK_ADD</code> par : <code>PCM_CAPTURE_ADD(IN_Capture, 0, PIPELINE_PCM_1)</code> <code>PCM_DUPLEX_ADD(OUT_Monitor, 1, PIPELINE_PCM_6, PIPELINE_PCM_7)</code>
M5 — <code>SectionGraph</code>	Inchangé : <code>dapm(PIPELINE_SINK_7, PIPELINE_DEMUX_6)</code> . Le flux data passe toujours par <code>MUXDEMUX6.0</code> → <code>BUF7.0</code> .

1.2 Rationale GLM

Pourquoi PIPE 7 autonome TIMER résout le bug -22 :

- Avec `SCHED_COMP = N_PCMC(7)` (HOST type local), PIPE 7 ne partage plus le `sched_comp` de PIPE 6 (`N_DAI_OUT(6)`)
- `pipeline_is_same_sched_comp(PIPE 6, PIPE 7) = false`
- Guard `pipeline-stream.c:505` bloque la propagation trigger cross-pipeline
- PIPE 7 est préparée et triggerée indépendamment (par `PCM_DUPLEX_ADD` via kernel DPCM)

Pourquoi `VIRTUAL_WIDGET out_drv` résout "no backend DAIs enabled" :

- Le widget `snd_soc_dapm_out_drv` apparaît dans le DAPM routing table du kernel
- ASoC DPCM trouve un endpoint BE virtuel pour router le FE capture HP_Monitor
- Pattern utilisé en production dans `pipe-detect.m4:93` pour le keyword-detect i.MX8MP

Pourquoi PCM_DUPLEX_ADD simplifie l'exposition ALSA :

- Un seul device ALSA OUT_Monitor avec 2 directions (playback + capture)
- Pattern upstream validé sof-smart-amplifier.m4:216
- Le kernel DPCM gère le scheduling des 2 streams indépendamment

1.3 Risques identifiés (à valider par investigation)

Risque	Description
R1 — VIRTUAL_WIDGET BE validity	Contradiction non résolue entre workers : GLM+Minimax disent que VIRTUAL_WIDGET(out_drv) sert de proxy BE pour ASoC DPCM. Gemini+claude-code disent que c'est juste un widget DAPM sans statut BE DAI link. Si gemini+claude-code ont raison, "no backend DAIs" persiste en V1.5_glm.
R2 — Drift DMA/TIMER	PIPE 6 en DMA (synchronisé SAI7) vs PIPE 7 en TIMER (Zephyr tick). Drift cumulatif possible sur longue durée. Buffer B0 PIPE 7 absorbe les différences. Si over/underrun, le MUXDEMUX drop silencieusement (overrun_permitted=true sur cross-pipeline sinks, mux.c:437-460).
R3 — Ordre de démarrage	Même pattern que masterlite : si PIPE 6 triggere avant que PIPE 7 soit prepared par le kernel, crash potentiel. PCM_DUPLEX_ADD devrait sérialiser via le même device, mais à valider.
R4 — Pattern non attesté	La combinaison (VIRTUAL_WIDGET + PIPE 7 autonome TIMER + PCM_DUPLEX_ADD + MUXDEMUX cross-pipeline) n'a pas d'équivalent direct en upstream. GLM s'appuie sur analogies partielles (masterlite, kwd, pipe-detect, pipe-volume-capture-sched template orphelin).

1.4 Plan de test

Si investigation approuve V1.5_glm :

1. Coder les 2 fichiers modifiés
2. Gate compile m4 + alsatplg : vérifier widgets + routes
3. Deploy sur board + reboot
4. T1 : aplay OUT_Monitor seul → doit fonctionner (retour V4.2 behavior)
5. T2 : arecord IN_Capture seul → inchangé Phase 1a.1
6. T3 : arecord OUT_Monitor seul sans aplay → doit ouvrir sans "no backend DAIs" (KEY TEST pour R1)
7. T4 : aplay OUT_Monitor + arecord OUT_Monitor simultané → capture contient le signal post-effets
8. T5 : Triplex mics + playback + HP_Monitor pendant 60s → pas de xrun

Si T3 ou T4 échoue → V1.5_kernel (patch DT + machine driver) comme fallback.

2 Ce qui change depuis V1.1

Investigation V1.1 (5 workers, qwen encore running) verdict : **4 GO fermes + 1 NO-GO (gemini-3-pro)**. Gemini a identifié un **bug d'ordre d'évaluation m4 critique** dans masterV42-hpmon.m4 non détecté par les 4 autres workers. V1.2 applique le fix.

2.1 Bug ordre m4 corrigé en V1.2

Problème V1.1 Dans `masterV42-hpmon.m4`, l'ordre était :

1. L376 : `PIPELINE_PCM_ADD PIPE 1`
2. L383 : `PIPELINE_PCM_ADD PIPE 6` (playback-tap)
3. L392 : `PIPELINE_PCM_ADD PIPE 7` ← utilise `PIPELINE_PLAYBACK_SCHED_COMP_6`
4. L400 : `DAI_ADD PIPE 1`
5. L405 : `DAI_ADD PIPE 6` ← **DÉFINIT** `PIPELINE_PLAYBACK_SCHED_COMP_6` (trop tard!)

À l'étape 3, la macro `PIPELINE_PLAYBACK_SCHED_COMP_6` n'existe **pas encore**. m4 passe donc la chaîne littérale. Le `W_PIPELINE(SCHED_COMP, ...)` de PIPE 7 génère `stream_name "PIPELINE_PLAYBACK_SCHED_COMP_6"` au lieu de `"SAI7.OUT"`. Le kernel topology loader ne trouve pas de match (phase 2 de résolution par nom) → `pipeline_is_same_sched_comp()` retourne false → guard `pipeline-stream.c:505` bloque → timeout -110 (rebelote V2.7).

Fix V1.2 Déplacer `DAI_ADD PIPE 6` avant `PIPELINE_PCM_ADD PIPE 7`. Pattern confirmé par `sof-imx8mp-wm89` où le `DAI_ADD` de la pipeline maître précède le `PIPELINE_PCM_ADD` piggyback.

Nouvel ordre :

1. `PIPELINE_PCM_ADD PIPE 1` (capture)
2. `PIPELINE_PCM_ADD PIPE 6` (playback-tap) → exporte `PIPELINE_SOURCE_6` et `PIPELINE_DEMUX_6`
3. `DAI_ADD PIPE 6` → définit `PIPELINE_PLAYBACK_SCHED_COMP_6 = N_DAI_OUT = SAI7.OUT`
4. `PIPELINE_PCM_ADD PIPE 7` avec `PIPELINE_PLAYBACK_SCHED_COMP_6` ← maintenant résolu correctement
5. `DAI_ADD PIPE 1` (ordre indifférent, PIPE 1 autonome)

2.2 Points V1.1 confirmés par 4 workers (kimi, minimax, claude-code, glm)

- B1 fix (export widget `N_MUXDEMUX(0)`) — validé unanime
- B2 fix (`W_PIPELINE` explicite PIPE 7) — validé unanime
- B3 fix (B5 supprimé) — validé unanime
- `dapm(BUF7.0, MUXDEMUX6.0)` cross-pipeline — confirmé pattern smart-amplifier
- `get_stream_index` match par `pipeline_id` (`idx=0` pour 6, `idx=1` pour 7) — OK
- `overrun_permitted=true` automatique sur sink cross-pipeline — OK NPU
- Ordre démarrage userspace recommandé : `aplay` avant `arecord` (caveat claude-code)

3 Ce qui change depuis V1

V1 soumise à investigation critic (5 workers convergents en 2027s, glm offline). Verdict : **architecture VIABLE avec 3 bugs critiques dans le squelette m4**. V1.1 applique les 3 corrections unanimement validées.

3.1 Bugs V1 corrigés en V1.1

#	Sévérité	Correction V1.1
B1	CRITICAL	PIPELINE_DEMUX_6 exportait N_BUFFER(5) (un buffer). Pattern upstream (pipe-smart-amplifier-playback.m4:135, pipe-amp-ref-capture.m4:109) exporte le widget MUXDEMUX N_MUXDEMUX(0). IPC3 autorise dapm(BUF, WIDGET) cross-pipeline mais refuse dapm(BUF, BUF) (commentaire historique masterlite.m4:81-83). V1.1 corrige.
B2	CRITICAL	PIPE 7 n'avait pas de W_PIPELINE explicite. Le piggyback via 13e arg PIPELINE_PCM_ADD définit la macro SCHED_COMP mais ne crée pas le widget scheduler. Pattern upstream (pipe-mastertap.m4:33-34, pipe-detect.m4:97) : toujours ajouter W_PIPELINE(SCHED_COMP, ...). V1.1 corrige.
B3	CRITICAL	Buffer B5 dans PIPE 6 redondant et buggué : B5.pipeline_id = 6 identique à B4.pipeline_id = 6 → collision dans mux.c:156-167 get_stream_index() qui matche les sinks par pipeline_id. Le MUXDEMUX ne saurait pas distinguer B4 (DAI) et B5 (tap), ROUTE_MATRIX(7) jamais appliquée. V1.1 supprime complètement B5 : le MUXDEMUX alimente directement BUF7.0 (PIPE 7) via SectionGraph cross-pipeline. Pattern upstream smart-amplifier validé (trouvaille claude-code).

3.2 Points V1 confirmés unanimement (pas de changement)

- **Piggyback syntax** : 13e arg PIPELINE_PLAYBACK_SCHED_COMP_6 + 12e arg SCHEDULE_TIME_DOMAIN_DMA corrects (référence sof-imx8mp-wm8960-kwd.m4:67-72)
- **MUXDEMUX dual-sink** intra-pipeline + cross-pipeline : viable (pattern smart-amplifier, MUX_MAX_STREAMS = 4 en IPC3)
- **ROUTE_MATRIX pipeline_id** : 6 pour sink DAI (intra), 7 pour sink cross (HP_Monitor)
 - corrects
- **SectionGraph** : dapm(PIPELINE_SINK_7, PIPELINE_DEMUX_6) devient dapm(BUF7.0, MUXDEMUX6.0) valide après B1
- **Latence tap** = 0 (demux copie synchrone au même tick)
- **Budget CPU** < 2% additionnel (Phase 1a.1 MVP @ 2ms validée, marge confortable)

4 Contexte — Phase 1a.1 validée

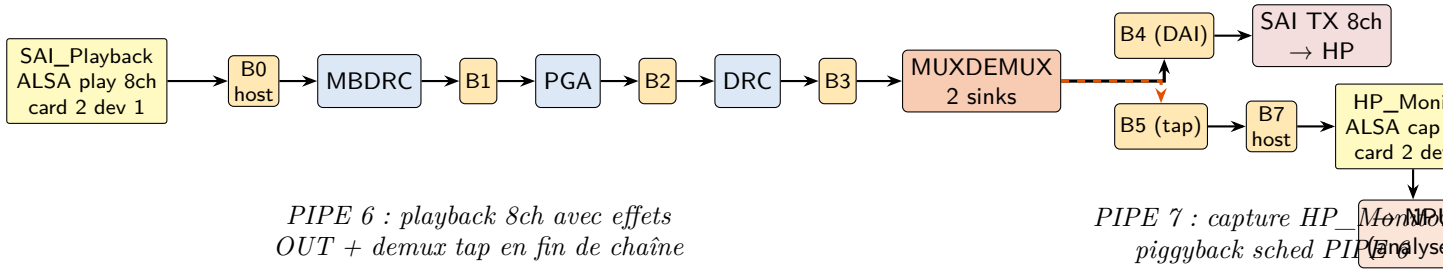
La Phase 1a.1 MVP V4.2 est **validée et déployée** sur board Debix 192.168.0.9 (commit sof fork 778740b68, branch feature/sdma-ap2ap-phase1) :

- 2 pipelines 8ch autonomes avec effets mbdrp + pga + drc chainés
- SAI7 TDM 8×32 @ 48 kHz ASYNC, period 2000 µs (4ms buffer par pipeline)
- PCMs exposés : SAI_Capture (card 2 device 0), SAI_Playback (card 2 device 1)
- Tests T0–T4 tous verts, sirène 5s 8ch audible sans clip
- Driver SAI intouché (validation 7 jours de travail utilisateur)

Phase 1a.2 : **ajouter un 3e PCM ALSA capture HP_Monitor** qui récupère le signal **post-effets OUT** (après drc de PIPE 6), pour analyse NPU et monitoring en parallèle du flux qui part vers les HP.

5 Architecture Phase 1a.2

5.1 Schéma cible



5.2 Mécanisme piggyback scheduling

Problème historique V2.7 : cross-pipeline en IPC3 échoue au trigger quand `sched_comp` diffère entre pipelines connectés (`pipeline-stream.c:505:if (!is_single_ppl && !is_same_sched) return 0`).

Solution V1 : **piggyback** la pipeline HP_Monitor (capture) sur le scheduler de la pipeline playback PIPE 6. Les deux partagent le même `sched_comp = N_DAI_OUT(6)` (exporté par `pipe-dai-playback.m4:24` sous le nom `PIPELINE_PLAYBACK_SCHED_COMP_6`).

Pattern upstream confirmé :

- `sof-ixmp8mp-wm8960-kwd.m4:72` : keyword-detect pipeline co-scheduled sur `PIPELINE_SCHED_COMP_1` (13e arg de `PIPELINE_PCM_ADD`)
- `sof-ixmp8mp-tac5212-masterlite.m4:51-56` : pattern historique avec `PIPELINE_SCHED_COMP_3`
- `pipeline.m4:84` : `define('SCHED_COMP', $13)`

5.3 Mécanisme MUXDEMUX en fin de pipeline playback

La pipeline playback existante (PIPE 6) doit être **modifiée** pour insérer un widget `W_MUXDEMUX` entre DRC et le buffer DAI. Le demux duplique le flux post-DRC vers 2 sinks :

- **Sink local** : buffer DAI (B4) → SAI TX (HP physiques)
- **Sink cross-pipeline** : exposé via `PIPELINE_DEMUX_6` → alimenté par une pipeline capture séparée (PIPE 7)

Attention bug V2.7 : le `PIPELINE_DEMUX_N` doit être **explicitement exporté** par le fichier pipeline playback. La V2.7 `masterlite.m4:88` référençait `PIPELINE_DEMUX_3` jamais défini, causant widget fantôme + timeout -110.

6 Phasage Phase 1a.2

Étape	Livrable
1a.2.a	Renommer <code>pipe-effects-playback.m4</code> → <code>pipe-effects-playback-tap.m4</code> avec insertion <code>W_MUX</code>
1a.2.b	Créer <code>pipe-effects-hp-monitor-capture.m4</code> (pipeline capture minimal : <code>buffer</code> → <code>PCM host</code>)
1a.2.c	Modifier <code>masterV42.m4</code> : ajouter PIPE 7 + SectionGraph cross-pipeline <code>dapm(PIPELINE_SINK_7, PIPELINE_SINK_8)</code>
1a.2.d	Tests T8-T12 : load, <code>HP_Monitor</code> visible, capture <code>HP_Monitor</code> solo, triplex <code>SAI_Capture</code> + <code>SAI_Playback</code>

Contraintes clés :

- Pipeline `HP_Monitor` sans effets : juste `buffer` + `W_PCM_CAPTURE`
- Pas de `DAI_ADD` pour PIPE 7 (pas de sortie hardware, juste tap interne)
- 13e arg `PIPELINE_PLAYBACK_SCHED_COMP_6` pour le piggyback
- Period 2000 μ s identique (cohérence scheduling avec PIPE 6)

7 Squelette m4 Phase 1a.2 V1

7.1 `sof-ixmp8mp-tac5212-pipe-effects-playback-tap.m4` (modifié)

Dérivé de la version Phase 1a.1 MVP, avec insertion `W_MUXDEMUX` entre DRC et le buffer DAI. Le demux a 2 sinks : le buffer B4 vers DAI et un buffer B5 exposé cross-pipeline via `PIPELINE_DEMUX_6`.

```
# Playback 8ch avec tap post-effets :
# HOST -> B0 -> MBDR -> B1 -> PGA -> B2 -> DRC -> B3 -> MUXDEMUX -> B4 -> DAI
#
#                                     |
#                                     +--> (cross-pipeline PIPE 7 HP_Monitor)

include('utils.m4')
include('buffer.m4')
include('pcm.m4')
include('dai.m4')
include('bytecontrol.m4')
include('mixercontrol.m4')
include('pipeline.m4')
include('multiband_drc.m4')
include('pga.m4')
include('drc.m4')
include('muxdemux.m4')
```

```

# ... (Controls PGA + MBDRC + DRC comme Phase 1a.1)

# MUXDEMUX route matrix : 1 source 8ch -> 2 sinks 8ch identiques
define(MY_DEMUX_CTRL, concat('demux_ctrl_', PIPELINE_ID))
define(DEMUX_priv, concat('demux_priv_', PIPELINE_ID))
include('demux_route_hpmon.m4') # à créer : identity 8ch x 2 sinks
C_CONTROLBYTES(MY_DEMUX_CTRL, PIPELINE_ID,
CONTROLBYTES_OPS(bytes, 258, 258, 258),
CONTROLBYTES_EXTOPS(258, 258, 258),
, , ,
CONTROLBYTES_MAX(, 304),
,
DEMUX_priv)

# Host endpoint
W_PCM_PLAYBACK(PCM_ID, SAI Playback, 2, 0, SCHEDULE_CORE)

# Widgets : MBDRC + PGA + DRC + MUXDEMUX
W_MULTIBAND_DRC(0, PIPELINE_FORMAT, 2, 2, SCHEDULE_CORE,
LIST(' ', "MY_MULTIBAND_DRC_CTRL"))
W_PGA(0, PIPELINE_FORMAT, DAI_PERIODS, 2, DEF_PGA_CONF, SCHEDULE_CORE,
LIST(' ', "PIPELINE_ID Master Playback Volume"))
W_DRC(0, PIPELINE_FORMAT, 2, 2, SCHEDULE_CORE,
LIST(' ', "MY_DRC_CTRL"))

# MUXDEMUX : 1 source (DRC) -> 2 sinks (B4 DAI, B5 tap)
# W_MUXDEMUX(index, mux_or_demux=1 demux, format, periods_sink, periods_source, core, kcontrols)
W_MUXDEMUX(0, 1, PIPELINE_FORMAT, 2, 2, SCHEDULE_CORE,
LIST(' ', "MY_DEMUX_CTRL"))

# Buffers : B0 host, B3 pré-demux, B4 DAI side, B5 tap side
W_BUFFER(0, COMP_BUFFER_SIZE(2,
COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
PLATFORM_HOST_MEM_CAP, SCHEDULE_CORE)
W_BUFFER(1, ..., PLATFORM_COMP_MEM_CAP, SCHEDULE_CORE)
W_BUFFER(2, ..., PLATFORM_COMP_MEM_CAP, SCHEDULE_CORE)
W_BUFFER(3, ..., PLATFORM_COMP_MEM_CAP, SCHEDULE_CORE)
W_BUFFER(4, COMP_BUFFER_SIZE(DAI_PERIODS,
COMP_SAMPLE_SIZE(DAI_FORMAT), PIPELINE_CHANNELS,
COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
PLATFORM_DAI_MEM_CAP, SCHEDULE_CORE)
# V1.1 fix B3 : B5 SUPPRIME (collision pipeline_id=6 avec B4 dans get_stream_index)
# Le MUXDEMUX alimente directement BUF7.0 de PIPE 7 via SectionGraph cross-pipeline.

# DAPM graph playback : host -> effets -> demux -> DAI
# Note : MUXDEMUX a 1 sink intra (B4 -> DAI) + 1 sink cross (BUF7.0 via SectionGraph)
P_GRAPH(pipe-effects-playback-tap, PIPELINE_ID,
LIST(' ',
'dapm(N_BUFFER(0), N_PCM(PCM_ID))',
'dapm(N_MULTIBAND_DRC(0), N_BUFFER(0))',
'dapm(N_BUFFER(1), N_MULTIBAND_DRC(0))',
'dapm(N_PGA(0), N_BUFFER(1))',
'dapm(N_BUFFER(2), N_PGA(0))',
'dapm(N_DRC(0), N_BUFFER(2))',
'dapm(N_BUFFER(3), N_DRC(0))',
'dapm(N_MUXDEMUX(0), N_BUFFER(3))',
'dapm(N_BUFFER(4), N_MUXDEMUX(0))'))
# V1.1 fix B3 : dapm(N_BUFFER(5), ...) SUPPRIME

# Exports V1.1 :
# SOURCE_6 = B4 (vers DAI via DAI_ADD)
# DEMUX_6 = WIDGET N_MUXDEMUX(0) (V1.1 fix B1 : widget, pas buffer)
indir('define', concat('PIPELINE_SOURCE_', PIPELINE_ID), N_BUFFER(4))
indir('define', concat('PIPELINE_DEMUX_', PIPELINE_ID), N_MUXDEMUX(0))
indir('define', concat('PIPELINE_PCM_', PIPELINE_ID), SAI Playback PCM_ID)

# PCM capabilities
PCM_CAPABILITIES(SAI Playback PCM_ID, ...)

```

7.2 sof-imx8mp-tac5212-pipe-hp-monitor-capture.m4 (nouveau)

Pipeline capture simplifié : **pas d'effets**, juste un buffer qui reçoit le flux cross-pipeline + endpoint PCM capture.


```

# HP_Monitor capture pipeline V1.5_glm :
# (cross-pipeline PIPELINE_DEMUX_6) -> B0 -> OUT_MONITOR_TAP (virtual) -> PCM host HP_Monitor
# V1.5_glm : pipeline AUTONOME TIMER (PAS de piggyback, contrairement V1.2)
# Sched_comp = N_PCMC(PCM_ID) HOST local -> is_same_sched=false avec PIPE 6 -> trigger ne traverse pas

include('utils.m4')
include('buffer.m4')
include('pcm.m4')
include('pipeline.m4')

# Host PCM endpoint
W_PCM_CAPTURE(PCM_ID, HP_Monitor, 0, 2, SCHEDULE_CORE)

# Buffer : B0 reçoit du cross-pipeline demux (PIPE 6)
W_BUFFER(0, COMP_BUFFER_SIZE(2,
    COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
    COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
    PLATFORM_HOST_MEM_CAP, SCHEDULE_CORE)

# V1.5_glm M3 : VIRTUAL_WIDGET out_drv pour résoudre kernel ASoC DPCM
# "no backend DAIs enabled". Pattern utilisé par pipe-detect.m4:93 (kwd i.MX8MP).
VIRTUAL_WIDGET(OUT_MONITOR_TAP PIPELINE_ID, out_drv, PIPELINE_ID)

# V1.5_glm M2 : W_PIPELINE avec SCHED_COMP LOCAL (N_PCMC type HOST)
# Pattern pipe-masterchain.m4:146-147 qui exporte PIPELINE_SCHED_COMP_X = N_PCMC(X).
# Ici on exporte pour PIPE 7 capture : PIPELINE_SCHED_COMP_7 = N_PCMC(PCM_ID).
indir('define', concat('PIPELINE_SCHED_COMP_', PIPELINE_ID), N_PCMC(PCM_ID))

W_PIPELINE(N_PCMC(PCM_ID), SCHEDULE_PERIOD, SCHEDULE_PRIORITY, SCHEDULE_CORE,
    SCHEDULE_TIME_DOMAIN, pipe_media_schedule_plat)

# Graph : flux capture = PCM_C <- B0 <- OUT_MONITOR_TAP (virtual)
# Le VIRTUAL_WIDGET apparaît comme endpoint BE côté kernel DAPM.
# Le buffer B0 reçoit EN PLUS du cross-pipeline via SectionGraph top-level.
P_GRAPH(pipe-hp-monitor-capture, PIPELINE_ID,
    LIST(' ',
        'dapm(N_PCMC(PCM_ID), N_BUFFER(0))',
        'dapm(N_BUFFER(0), OUT_MONITOR_TAP PIPELINE_ID)'))

# Exports
indir('define', concat('PIPELINE_SINK_', PIPELINE_ID), N_BUFFER(0))
indir('define', concat('PIPELINE_PCM_', PIPELINE_ID), HP_Monitor PCM_ID)

PCM_CAPABILITIES(HP_Monitor PCM_ID, CAPABILITY_FORMAT_NAME(PIPELINE_FORMAT),
    PCM_MIN_RATE, PCM_MAX_RATE, 2, PIPELINE_CHANNELS, 2, 16,
    192, 16384, 65536, 65536)

```

7.3 sof-imx8mp-tac5212-masterV42-hpmon.m4 (top-level étendu)

```

# Topology V4.2 + Phase 1a.2 V1.5_glm : HP_Monitor tap post-effets OUT (m4-only)
# 3 pipelines : PIPE 1 capture mics, PIPE 6 playback effets + MUXDEMUX tap, PIPE 7 HP_Monitor capture
# V1.5_glm : PIPE 7 AUTONOME TIMER (pas piggyback DAI) + VIRTUAL_WIDGET + PCM_DUPLEX_ADD

include('utils.m4')
include('dai.m4')
include('pipeline.m4')
include('sai.m4')
include('pcm.m4')
include('buffer.m4')
include('common/tlv.m4')
include('sof/tokens.m4')
include('platform/imx/imx8.m4')

# PIPE 1 : Capture 8ch avec effets IN (inchangé Phase 1a.1)
PIPELINE_PCM_ADD(sof/sof-imx8mp-tac5212-pipe-effects-capture.m4,
    1, 0, 8, s32le,
    2000, 0, 0,
    48000, 48000, 48000,
    SCHEDULE_TIME_DOMAIN_DMA)

# PIPE 6 : Playback 8ch avec effets OUT + MUXDEMUX tap
PIPELINE_PCM_ADD(sof/sof-imx8mp-tac5212-pipe-effects-playback-tap.m4,
    6, 1, 8, s32le,

```

```

2000, 0, 0,
48000, 48000, 48000,
SCHEDULE_TIME_DOMAIN_DMA)

# V1.2 fix ordre m4 : DAI_ADD PIPE 6 AVANT PIPELINE_PCM_ADD PIPE 7
# pour définir PIPELINE_PLAYBACK_SCHED_COMP_6 utilisé par PIPE 7.
# Pattern upstream : sof-imx8mp-wm8960-kwd.m4 (DAI_ADD maître avant piggyback)
DAI_ADD(sof/pipe-dai-playback.m4,
        6, SAI, 7, tac5212-hifi,
        PIPELINE_SOURCE_6, 2, s32le,
        2000, 0, 0, SCHEDULE_TIME_DOMAIN_DMA)

# PIPE 7 : HP_Monitor capture, AUTONOME TIMER (V1.5_glm M1 : pas de piggyback)
# 12e arg SCHEDULE_TIME_DOMAIN_TIMER (PIPE 7 n'a pas de DAI → pas de DMA source)
# PAS de 13e arg : sched_comp local (N_PCMC(PCM_ID)) défini dans pipe-hp-monitor-capture.m4
# is_same_sched(PIPE 6, PIPE 7) = false → trigger ne traverse pas cross-pipeline
PIPELINE_PCM_ADD(sof/sof-imx8mp-tac5212-pipe-hp-monitor-capture.m4,
        7, 2, 8, s32le,
        2000, 0, 0,
        48000, 48000, 48000,
        SCHEDULE_TIME_DOMAIN_TIMER)

# DAI_ADD PIPE 1 (ordre indifférent, PIPE 1 autonome)
DAI_ADD(sof/pipe-dai-capture.m4,
        1, SAI, 7, tac5212-hifi,
        PIPELINE_SINK_1, 2, s32le,
        2000, 0, 0, SCHEDULE_TIME_DOMAIN_DMA)

# PCM ALSA exports V1.5_glm (M4) : renommage + PCM_DUPLEX_ADD
# IN_Capture (card device 0, capture only) = les 8 mics post-effets IN
# OUT_Monitor (card device 1, duplex) = aplay: DAW→effets OUT→HP
# arecord: tap post-effets OUT pour NPU
PCM_CAPTURE_ADD(IN_Capture, 0, PIPELINE_PCM_1)
PCM_DUPLEX_ADD(OUT_Monitor, 1, PIPELINE_PCM_6, PIPELINE_PCM_7)

# Graph cross-pipeline : tap PIPE 6 demux -> PIPE 7 sink buffer (inchangé V1.2)
SectionGraph."pipe-tap-hpmon" {
    index "0"
    lines [
        dapm(PIPELINE_SINK_7, PIPELINE_DEMUX_6)
    ]
}

# SAI7 DAI config (inchangé Phase 1a.1)
DAI_CONFIG(SAI, 7, 0, tac5212-hifi,
        SAI_CONFIG(DSP_A, SAI_CLOCK(mclk, 12288000, codec_mclk_in),
        SAI_CLOCK(bclk, 12288000, codec_consumer),
        SAI_CLOCK(fsyc, 48000, codec_consumer),
        SAI_TDM(8, 32, 255, 255),
        SAI_CONFIG_DATA(SAI, 7, 0)))

```

7.4 m4/demux_route_hpmon.m4 (nouveau blob route matrix)

Duplication identité 8 canaux vers 2 sinks. Respecte mux_mix_check() de sof/src/audio/mux/mux.c:44 : **pas de sommation** entre streams (popcount(mask) == 1 par byte).

```

divert(-1)

# demux_route_hpmon.m4 Phase 1a.2
# 1 source 8ch -> 2 sinks 8ch (identity duplication)
# stream 0 : sink vers buffer DAI (pipeline 6 interne) -> SAI TX
# stream 1 : sink vers buffer tap (cross-pipeline 7) -> HP_Monitor

define('matrix_to_dai', 'ROUTE_MATRIX(6,
    'BITS_TO_BYTE(1, 0, 0, 0, 0, 0, 0, 0)',
    'BITS_TO_BYTE(0, 1, 0, 0, 0, 0, 0, 0)',
    'BITS_TO_BYTE(0, 0, 1, 0, 0, 0, 0, 0)',
    'BITS_TO_BYTE(0, 0, 0, 1, 0, 0, 0, 0)',
    'BITS_TO_BYTE(0, 0, 0, 0, 1, 0, 0, 0)',
    'BITS_TO_BYTE(0, 0, 0, 0, 0, 1, 0, 0)',
    'BITS_TO_BYTE(0, 0, 0, 0, 0, 0, 1, 0)',
    'BITS_TO_BYTE(0, 0, 0, 0, 0, 0, 0, 1)')')

define('matrix_to_hpmon', 'ROUTE_MATRIX(7,

```

```

'BITS_TO_BYTE(1, 0, 0, 0, 0, 0, 0, 0)',
'BITS_TO_BYTE(0, 1, 0, 0, 0, 0, 0, 0)',
'BITS_TO_BYTE(0, 0, 1, 0, 0, 0, 0, 0)',
'BITS_TO_BYTE(0, 0, 0, 1, 0, 0, 0, 0)',
'BITS_TO_BYTE(0, 0, 0, 0, 1, 0, 0, 0)',
'BITS_TO_BYTE(0, 0, 0, 0, 0, 1, 0, 0)',
'BITS_TO_BYTE(0, 0, 0, 0, 0, 0, 1, 0)',
'BITS_TO_BYTE(0, 0, 0, 0, 0, 0, 0, 1)')')

divert(0)dn1
MUXDEMUX_CONFIG(DEMUX_priv, 2,
LIST_NONEWLINE('', 'matrix_to_dai,', 'matrix_to_hpmon'))

```

8 Tests Phase 1a.2

1. **T8 Gate compile** : m4 | alsatplg sans erreur, aucun widget orphelin. Vérifier alsatplg -dump | grep MUXDEMUX présent, grep "PIPELINE_DEMUX_6" masterV42-hpmon.conf présent avant SectionGraph.
2. **T9 Load** : dmesg propre (pas de -22 ni -110). 3 PCMs visibles : SAI_Capture (dev 0), SAI_Playback (dev 1), HP_Monitor (dev 2).
3. **T10 HP_Monitor solo** : arecord -D plughw:softac5212tdm,2 -c 8 -f S32_LE -r 48000 -d 3 /tmp/hpmon_solo.wav → 4.6MB mais **silence** si playback pas actif (pas de signal à tap). Vérifier juste l'ouverture sans EIO.
4. **T11 HP_Monitor avec playback** : aplay -D plughw:softac5212tdm,1 siren.wav & puis arecord -D plughw:softac5212tdm,2 -c 8 -f S32_LE -r 48000 -d 5 /tmp/hpmon_play.wav → doit contenir la sirène post-effets (DRC + PGA appliqués).
5. **T12 Triplex complet** : arecord -D plughw:softac5212tdm,0 (SAI_Capture) + aplay -D plughw:softac5212tdm,1 (SAI_Playback) + arecord -D plughw:softac5212tdm,2 (HP_Monitor) simultanés. Les 3 doivent tourner sans xrun 10s minimum.
6. **T13 Null test post-effets** : comparer /tmp/hpmon_play.wav contenu avec le tone original après DRC bypass (via amixer cset du switch DRC) → confirme le tap est bien post-DRC (pas pre-DRC).

9 Questions pour l'investigation critic V1

1. **Q1 Piggyback scheduling syntax** : le 13e arg PIPELINE_PLAYBACK_SCHED_COMP_6 de PIPELINE_PCM_ADD est-il la bonne syntaxe ? Faut-il que PIPE 7 passe aussi le 12e arg SCHEDULE_TIME_DOMAIN_DMA ou hérite-t-il via le piggyback ? Vérifier pipeline.m4 + sof-smart-amplifier.m4 + sof-imx8mp-wm8960-kwd.m4.
2. **Q2 MUXDEMUX 2 sinks même pipeline + 1 cross-pipeline** : le W_MUXDEMUX avec mode demux=1 peut-il avoir simultanément un sink INTRA-pipeline (B4 → DAI dans PIPE 6) ET un sink cross-pipeline (B5 exposé via PIPELINE_DEMUX_6 → PIPE 7) ? Le pattern upstream sof-smart-amplifier l'utilise dans quel sens exact ?
3. **Q3 Export PIPELINE_DEMUX_N** : pour éviter le bug V2.7 (PIPELINE_DEMUX_3 fantôme), le squelette V1 exporte PIPELINE_DEMUX_6 = N_BUFFER(5). Est-ce le bon widget à exporter (buffer, pas MUXDEMUX widget lui-même) ? Confirmer avec pipe-amp-ref-capture.m4.
4. **Q4 MUXDEMUX route matrix** : le blob demux_route_hpmon.m4 avec 2 streams d'identité 8ch (vers sinks pipe_id 6 et 7) respecte-t-il mux_mix_check ? Les ROUTE_MATRIX(pipeline_id, ...) doivent-ils utiliser les pipeline_id de destination (6 et 7) ?
5. **Q5 Pipeline 7 sans W_PIPELINE** : PIPE 7 ne reçoit pas de DAI_ADD (pas de DAI réel). Sans DAI_ADD, le scheduler widget W_PIPELINE n'est pas créé. Le piggyback PIPELINE_PLAYBACK_SCHED_COMP_6 compense-t-il ? Ou faut-il ajouter un W_PIPELINE explicite avec le sched_comp partagé ?
6. **Q6 Latence tap** : le tap post-DRC ajoute-t-il une latence additionnelle vs le signal qui va au DAI ? Le MUXDEMUX copie-t-il au même tick d'horloge ou avec décalage ?

7. **Q7 Risques CPU** : avec 3 pipelines actifs (PIPE 1, 6, 7) au lieu de 2, le budget CPU HiFi4 @ 2ms tient-il encore ? PIPE 7 est minimaliste (juste 1 buffer + 1 PCM), donc marginal, mais à confirmer.

10 Livrables Phase 1a.2

- `sof/tools/topology/topology1/sof-imx8mp-tac5212-masterV42-hpmon.m4` (top-level 3 pipelines + SectionGraph)
- `sof/tools/topology/topology1/sof/sof-imx8mp-tac5212-pipe-effects-playback-tap.m4` (playback avec MUXDEMUX)
- `sof/tools/topology/topology1/sof/sof-imx8mp-tac5212-pipe-hp-monitor-capture.m4` (capture piggyback)
- `sof/tools/topology/topology1/m4/demux_route_hpmon.m4` (route matrix identity $8ch \times 2$)
- Le fichier original `pipe-effects-playback.m4` reste intact (Phase 1a.1 MVP sans tap peut être redéployé en rollback)

11 Protocole

- Aucune modification SOF source sans `critic_analyze` préalable (fenêtre 2000s).
- Gate `m4 | alsatplg` OBLIGATOIRE avant deploy sur board.
- Backup `tplg V4.2 Phase 1a.1` (déjà présent : `sof-imx8mp-tac5212.tplg.bak-v27-preV42 + .tplg` actuel 13819 bytes validé).
- Si T8 échoue → retour `critic_analyze` avec erreur exacte avant modif.
- Driver SAI `sof/src/drivers/imx/sai.c` INTOUCHABLE (7 jours de travail utilisateur).